

# ESWIN

## PYESSDK快速入门指南

---

Rev 1.1

2026-07-29

## 声明

### 知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

### 责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

## 修订记录

| 版本号  | 日期         | 说明                                                              |
|------|------------|-----------------------------------------------------------------|
| v1.1 | 2026-07-29 | 更新 wheel 拆包说明，明确 es_py 核心包与 pyessdk-eswin-sample 示例包可同时安装且无文件覆盖 |
| v1.0 | 2026-05-12 | 重构文档结构                                                          |
| v0.1 | 2025-12-30 | 初始版本                                                            |

目录

声明 1

    知识产权声明 1

    责任限制 1

修订记录 2

1. 快速入门指南 5

    1.1. 简介 5

    1.2. 主要功能模块 5

    1.3. 环境要求 5

    1.4. ESSDK 安装 5

        1.4.1. 安装 SDK DEB 包 5

        1.4.2. 验证 SDK 安装 6

    1.5. PYESSDK 安装 6

        1.5.1. 在线源 6

        1.5.2. 安装步骤 6

        1.5.3. 快速验证环境 7

    1.6. 第一个程序 7

    1.7. 运行示例程序 8

2. 公共定义模块 9

    2.1. 模块概述 9

    2.2. 常量定义 9

        2.2.1. 返回值常量 9

        2.2.2. 特殊值定义 9

        2.2.3. 系统限制常量 9

    2.3. 枚举类型 10

        2.3.1. 缓存模式枚举 (SYS\_CACHE\_MODE\_E) 10

        2.3.2. 像素格式枚举 (PIXEL\_FORMAT\_E) 10

        2.3.3. VB 用户模块 ID (VB\_UID\_E) 11

        2.3.4. 系统模块 ID (MOD\_ID\_E) 11

        2.3.5. 数据类型枚举 (DATA\_TYPE\_E) 12

    2.4. 数据结构 12

        2.4.1. VB 内存池配置结构体 (VB\_POOL\_CONFIG\_S) 12

        2.4.2. VB 系统配置结构体 (VB\_CONFIG\_S) 12

        2.4.3. 通道结构体 (CHN\_S) 13

        2.4.4. 设备缓冲区结构体 (ES\_DEV\_BUF\_S) 14

        2.4.5. NPU 张量相关结构体 15

        2.4.6. 视频帧结构体 (VIDEO\_FRAME\_S) 16

        2.4.7. 视频帧信息结构体 (VIDEO\_FRAME\_INFO\_S) 16

    2.5. 枚举类型 API 17

        2.5.1. 错误级别枚举 (ERR\_LEVEL\_E) 17

        2.5.2. 错误码枚举 (EN\_ERR\_CODE\_E) 17

        2.5.3. 载荷类型枚举 (PAYLOAD\_TYPE\_E) 17

        2.5.4. 视频相关枚举 (像素格式) 18

        2.5.5. 操作模式和控制枚举 18

        2.5.6. VB 源类型枚举 (VB\_SOURCE\_E) 18

        2.5.7. 系统配置相关结构体 19

    2.6. 工具函数和编程方法 19

        2.6.1. 内存模块函数 (memory) 19

        2.6.2. 内存对齐工具函数 20

        2.6.3. 图像缓冲区大小计算函数 20

        2.6.4. 颜色处理工具函数 21

        2.6.5. 字节序和分数工具函数 21

        2.6.6. 实用编程示例 22

|                                              |           |
|----------------------------------------------|-----------|
| <b>3. Sample 示例</b>                          | <b>23</b> |
| 3.1. 功能模块                                    | 23        |
| 3.2. 软件包结构                                   | 23        |
| 3.3. 示例模块概览                                  | 23        |
| 3.4. 示例分类                                    | 23        |
| 3.5. 内存管理示例                                  | 23        |
| 3.5.1. memory_test.py - 内存测试套件               | 23        |
| 3.5.2. memory_test2.py - 进程间内存传输测试           | 24        |
| 3.6. 内存复制示例                                  | 25        |
| 3.6.1. memcp_test.py - DMA 内存复制演示            | 25        |
| 3.7. NPU 推理示例                                | 25        |
| 3.7.1. npu_runtime_sample - YOLO 多流检测（多模块联动） | 25        |
| 3.8. 视频处理示例                                  | 32        |
| 3.8.1. py_vdec - 视频解码示例                      | 32        |
| 3.8.2. py_venc - 视频编码示例                      | 33        |
| 3.8.3. sample_vo - 视频输出示例                    | 33        |
| 3.8.4. py_vps - 视频处理示例                       | 34        |
| 3.9. 复合视频流水线示例                               | 36        |
| 3.9.1. vdec_venc - 解码+编码流水线                  | 36        |
| 3.9.2. vdec_vps_venc - 解码+处理+编码流水线           | 36        |
| 3.9.3. vdec_vps_vo - 解码+处理+显示流水线             | 37        |
| <b>4. 故障排除</b>                               | <b>38</b> |
| 4.1. 安装与环境配置问题                               | 38        |
| 4.1.1. 问题 1: Python 模块导入失败                   | 38        |
| 4.1.2. 问题 2: 缺少底层系统依赖 (SO 库)                 | 38        |
| 4.1.3. 问题 3: Python 版本不兼容                    | 39        |
| 4.2. 运行时问题                                   | 39        |
| 4.2.1. 问题 4: 设备权限不足                          | 39        |
| 4.2.2. 问题 5: 硬件设备不可用                         | 39        |
| 4.3. 调试与诊断工具                                 | 40        |
| 4.3.1. 综合调试脚本                                | 40        |
| 4.4. 常用命令速查表                                 | 41        |
| 4.4.1. Python 环境检查                           | 41        |
| 4.4.2. Python 包管理                            | 41        |
| 4.4.3. 系统状态检查                                | 41        |
| 4.4.4. 快速测试命令                                | 41        |

# 1. 快速入门指南

## 1.1. 简介

本文档旨在帮助开发者快速上手 ESWIN PyESSDK。内容涵盖从环境准备、SDK 安装到运行第一个程序的完整流程。

es\_py 是基于 pybind11 技术开发的 ESSDK 的 Python 接口库，专为 EIC77 系列 AI SoC 设计。它通过封装底层 C/C++ SDK，使开发者能够使用 Python 语言直接调用硬件加速能力，实现异构计算。

从 v1.1 起，es\_py wheel 仅包含核心 Python 绑定；示例源码和运行数据由 pyessdk-eswin-sample wheel 独立提供。两个 wheel 可安装到同一 Python 环境，安装路径不重叠。

## 1.2. 主要功能模块

| 模块类型  | 功能模块         | 主要功能                      |
|-------|--------------|---------------------------|
| 基础系统  | es_py.memory | VB 内存池、系统内存分配             |
|       | es_py.sys    | 系统初始化、设备管理                |
|       | es_py.common | 公共定义、枚举类型、结构体             |
| 视频处理  | es_py.vdec   | 视频解码 (H.264/H.265/MJPEG)  |
|       | es_py.venc   | 视频编码 (H.264/H.265/MJPEG)  |
|       | es_py.vps    | 视频处理系统 (缩放、裁剪、格式转换)       |
|       | es_py.vo     | 视频输出 (显示控制、多路输出)          |
| AI 推理 | es_py.npu    | NPU 神经网络推理 (模型加载、同步/异步推理) |
| 性能加速  | es_py.memcp  | 内存复制加速 (DMA 传输)           |
|       | es_py.ak     | 加速工具包 (硬件加速)              |

## 1.3. 环境要求

- **硬件**：ESWIN EIC77 系列开发板，至少 500MB 存储空间（用于 Python 环境和 SDK）
- **系统**：Linux 系统，Python 3.12.6（必须严格匹配）

## 1.4. ESSDK 安装

### 1.4.1. 安装 SDK DEB 包

在安装 Python SDK 之前，必须先安装 ESWIN SDK 的原生 DEB 包。

```
# 更新包管理器
sudo apt update

# 安装核心 SDK 包
sudo apt install -y \
    es-sdk-common \
    es-sdk-sys \
    es-sdk-memory \
    es-sdk-video \
    es-sdk-npu \
    es-sdk-memcp \
    es-sdk-ak \
    es-sdk-numa \
    es-sdk-log
```

```

es-sdk-video-utils \
es-sdk-sample-video

# 安装其他基础依赖包
sudo apt install -y gcc g++ cmake ffmpeg libavformat-dev libavcodec-dev libavdevice-dev \
    libavutil-dev libavfilter-dev libswscale-dev libswresample-dev pkg-config python3-dev python3.12-venv

# 验证安装
dpkg -l | grep es-sdk

```

### 1.4.2. 验证 SDK 安装

```

# 检查库文件
ls /usr/lib/libes_*.so 2>/dev/null | head -5 && echo "SDK 库文件安装正常" || echo "警告：SDK 库文件未找到"

# 验证库链接
/sbin/ldconfig -p | grep libes_ | head -5 && echo "SDK 库链接正常" || echo "警告：SDK 库链接异常"

# 检查设备文件
ls /dev/es_* 2>/dev/null && echo "设备文件存在" || echo "警告：设备文件未找到"

```

## 1.5. PYESSDK 安装

**目标：**在板端 Python 3.12.6 环境中，本地创建 `venv` 并通过在线源安装依赖、`es_py` 核心包和可选示例包。

**适用：**当前 `es_py-1.12.2` 核心 wheel 与 `pyessdk-eswin-sample-1.0.0` 示例 wheel 交付方式。

### 1.5.1. 在线源

<https://archive.eswincomputing.com/pypi/simple/>

### 1.5.2. 安装步骤

```

# 本地创建并激活虚拟环境
python3 -m venv ~/eswin_venv
source ~/eswin_venv/bin/activate

# 1. 先安装 numpy
pip3 install --index-url https://archive.eswincomputing.com/pypi/simple/ numpy

# 2. 再安装 opencv4
pip3 install --index-url https://archive.eswincomputing.com/pypi/simple/ opencv4

# 3. 安装 av
pip3 install --index-url https://archive.eswincomputing.com/pypi/simple/ av

# 4. 最后安装 es_py
pip3 install --index-url https://archive.eswincomputing.com/pypi/simple/ es_py

# 5. 如需运行示例，安装独立示例包
pip3 install --index-url https://archive.eswincomputing.com/pypi/simple/ pyessdk-eswin-sample

```

### 提示

opencv4 安装后通过 `import cv2` 调用。

### 1.5.3. 快速验证环境

完成 wheel 安装后，建议执行以下验证：

```
# 检查 Python 版本（必须是 3.12.6）
python3 --version
which python3 # 应指向 ~/eswin_venv/bin/python3

# 测试核心模块导入
python3 -c "
from es_py import memory, common, npu, sys as es_sys
print('核心模块导入成功')
"
```

## 1.6. 第一个程序

创建一个简单的 Python 脚本来测试 py-essdk 的基本功能：

```
#!/usr/bin/env python3
"""
ESWIN SDK Python 快速开始示例
测试基本内存管理功能
"""

from es_py import memory, common

def test_basic_memory():
    """测试基本内存功能"""
    print("开始测试内存分配...")

    try:
        # 分配 4KB 内存
        mem_fd = memory.memory_alloc(
            common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED,
            "sys",
            "system",
            4096
        )
        print(f"内存分配成功: fd=0x{mem_fd:x}")

        # 内存映射
        mv, vaddr = memory.memory_map(mem_fd, 4096,
            common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED)
        print(f"内存映射成功: vaddr={vaddr}")

        # 清理资源
        memory.memory_unmap(vaddr, 4096)
        memory.memory_free(mem_fd)
        print("内存清理完成")
        print("基本内存测试成功！")

    except Exception as e:
        print(f"测试失败: {e}")
        return False
```

```
    return True

if __name__ == "__main__":
    success = test_basic_memory()
    exit(0 if success else 1)
```

保存为 `test_memory.py`，然后运行：

```
python3 test_memory.py
```

## 1.7. 运行示例程序

`es_py` 核心 wheel 不再内置示例程序。安装 `pyessdk-eswin-sample` 后，可通过 Python 模块方式运行示例：

```
# 1. 运行基础内存测试示例
python3 -m pyessdk_eswin_sample.memory_sample.memory_test

# 2. 运行内存复制示例
python3 -m pyessdk_eswin_sample.memcp_sample.memcp_test

# 3. 运行视频解码示例（需要硬件支持）
python3 -m pyessdk_eswin_sample.video_sample.vdec.py_vdec

# 4. 运行视频输出示例（需要硬件支持和 HDMI 显示器）
python3 -m pyessdk_eswin_sample.video_sample.vo.sample_vo

# 5. 运行 NPU 推理示例（需要模型文件和配置）
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream --help
```

## 2. 公共定义模块

公共定义模块是 ESWIN SDK Python API 的基础模块，提供了所有其他模块共享的数据类型、枚举常量、结构体定义和通用工具函数。该模块确保了整个 API 体系的类型安全和接口一致性。

### 提示

具体功能模块的详细 Python API 调用方法、参数配置及示例代码，请分别参阅对应的各模块开发者手册。

### 2.1. 模块概述

- **作用**：提供全局共享的数据类型和常量定义
- **依赖**：无（基础模块）
- **被依赖**：所有其他模块都依赖此模块
- **底层映射**：对应原生 SDK 头文件 `es_common.h`、`es_defines.h` 等

### 2.2. 常量定义

#### 2.2.1. 返回值常量

```
from es_py import common

# 函数返回值（从底层 es_type.h 导入）
ES_SUCCESS = 0      # 操作成功
ES_FAILURE = -1     # 操作失败
ES_NULL = 0         # 空指针值
```

### 提示

虽然 SDK 提供了这些返回值常量，但 Python 封装的 API 在失败时会**自动抛出异常**，因此通常不需要手动检查返回值。这些常量主要用于与底层 C API 交互时的兼容性。

#### 2.2.2. 特殊值定义

```
# 无效值定义（从底层 es_common.h 导入）
ES_INVALID_CHN = -1  # 无效通道 ID
ES_INVALID_DEV = -1  # 无效设备 ID
ES_INVALID_HANDLE = -1 # 无效句柄
ES_INVALID_VALUE = -1 # 无效值

# VB 内存池相关（从底层 es_vb_user.h 导入）
VB_INVALID_POOLID = 0xFFFFFFFF # 无效 VB 池 ID（-1 的无符号表示）
```

#### 2.2.3. 系统限制常量

```
# 内存相关限制（从底层 es_defines.h 和 es_vb_user.h 导入）
ES_MAX_MMZ_NAME_LEN = 64 # MMZ 区域名称最大长度
ES_VB_MAX_POOLS = 768   # VB 内存池最大数量

# 模块特定限制（从底层 es_defines.h 导入，不同模块限制不同）
ES_VENC_MAX_CHN_NUM = 128 # VENC 最大通道数
ES_VDEC_MAX_GRP_NUM = 128 # VDEC 最大组数
ES_VPS_MAX_GRP_NUM = 256 # VPS 最大组数
```

```
ES_VPS_MAX_CHN_NUM = 3  # VPS 每组最大通道数
ES_VO_MAX_CHN_NUM = 64  # VO 最大通道数
```

### 提示

所有常量都通过 pybind11 从底层 SDK 头文件导出到 Python。不同硬件模块有各自的设备和通道数限制，具体限制请参考各模块章节。

## 2.3. 枚举类型

### 2.3.1. 缓存模式枚举 (SYS\_CACHE\_MODE\_E)

```
from es_py.common import SYS_CACHE_MODE_E

# 内存缓存模式
SYS_CACHE_MODE_NOCACHE = 0  # 非缓存模式（数据一致性优先）
SYS_CACHE_MODE_CACHED = 1  # 缓存模式（性能优先）
SYS_CACHE_MODE_LLC = 2  # LLC 缓存模式（NPU 专用）

# 使用示例
def allocate_cached_memory():
    return memory.memory_alloc(
        cache_type=common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED,
        p_str_mmb="cached_buffer",
        str_zone="mmz_nid_0_part_0",
        len=1024*1024
    )
```

### 2.3.2. 像素格式枚举 (PIXEL\_FORMAT\_E)

```
from es_py.common import PIXEL_FORMAT_E

# YUV 平面格式
PIXEL_FORMAT_YUV400 = 0  # 灰度图（仅 Y 分量）
PIXEL_FORMAT_YV12 = 1  # YV12 平面格式
PIXEL_FORMAT_I420 = 2  # I420 平面格式
PIXEL_FORMAT_YUV444P = 3  # YUV444 平面格式

# YUV 半平面格式
PIXEL_FORMAT_NV12 = 4  # NV12 格式 (Y + UV 交错)
PIXEL_FORMAT_NV21 = 5  # NV21 格式 (Y + VU 交错)
PIXEL_FORMAT_NV16 = 6  # NV16 格式
PIXEL_FORMAT_NV61 = 7  # NV61 格式

# RGB 格式
PIXEL_FORMAT_RGB565 = 8  # RGB565 (16 位)
PIXEL_FORMAT_BGR565 = 9  # BGR565 (16 位)
PIXEL_FORMAT_RGB888 = 10  # RGB888 (24 位)
PIXEL_FORMAT_BGR888 = 11  # BGR888 (24 位)
PIXEL_FORMAT_ARGB8888 = 12  # ARGB8888 (32 位)
PIXEL_FORMAT_ABGR8888 = 13  # ABGR8888 (32 位)
```

### 2.3.3. VB 用户模块 ID (VB\_UID\_E)

```
from es_py.common import VB_UID_E

# 硬件模块标识符 (用于 IOVA 分配)
VB_UID_PRIVATE = 0 # 私有模块
VB_UID_COMMON = 1 # 公共模块
VB_UID_VI = 2 # 视频输入模块
VB_UID_VO = 3 # 视频输出模块
VB_UID_VPS = 4 # 视频处理系统模块
VB_UID_VENC = 5 # 视频编码器模块
VB_UID_VDEC = 6 # 视频解码器模块
VB_UID_HAE = 7 # HAE 模块
VB_UID_USER = 8 # 用户模块

# IOVA 分配示例
def allocate_venc_iova(mem_fd):
    iova_capsule = es_py.memory.video_buffer_alloc_iova(
        mem_fd,
        common.VB_UID_E.VB_UID_VENC
    )
    return iova_capsule
```

### 2.3.4. 系统模块 ID (MOD\_ID\_E)

```
from es_py.common import MOD_ID_E

# 系统模块标识
ES_ID_VB = 0 # VB 内存管理模块
ES_ID_SYS = 1 # 系统管理模块
ES_ID_VDEC = 2 # 视频解码模块
ES_ID_VPS = 3 # 视频处理系统
ES_ID_VENC = 4 # 视频编码模块
ES_ID_VO = 5 # 视频输出模块
ES_ID_VI = 6 # 视频输入模块
ES_ID_AIO = 7 # 音频 IO 模块
ES_ID_AI = 8 # 音频输入模块
ES_ID_AO = 9 # 音频输出模块
ES_ID_AENC = 10 # 音频编码模块
ES_ID_ADEC = 11 # 音频解码模块
ES_ID_USER = 12 # 用户模块
ES_ID_GDC = 13 # GDC 模块
ES_ID_NPU = 14 # NPU 推理模块
ES_ID_DSP = 15 # DSP 处理模块
ES_ID_BMS = 16 # BMS 模块

# 通道绑定示例
def bind_vdec_to_vps(vdec_chn, vps_grp):
    src_chn = CHN_S()
    src_chn.modId = MOD_ID_E.ES_ID_VDEC
    src_chn.devId = 0
    src_chn.chnId = vdec_chn

    dst_chn = CHN_S()
    dst_chn.modId = MOD_ID_E.ES_ID_VPS
    dst_chn.devId = vps_grp
    dst_chn.chnId = 0

    return es_py.sys.bind(src_chn, dst_chn)
```

### 2.3.5. 数据类型枚举 (DATA\_TYPE\_E)

```
from es_py.common import DATA_TYPE_E

# NPU 数据类型
DATA_TYPE_UINT8 = 0    # 无符号 8 位整数
DATA_TYPE_INT8 = 1     # 有符号 8 位整数
DATA_TYPE_UINT16 = 2   # 无符号 16 位整数
DATA_TYPE_INT16 = 3    # 有符号 16 位整数
DATA_TYPE_UINT32 = 4   # 无符号 32 位整数
DATA_TYPE_INT32 = 5    # 有符号 32 位整数
DATA_TYPE_FLOAT16 = 6  # 16 位浮点数
DATA_TYPE_FLOAT32 = 7  # 32 位浮点数
```

## 2.4. 数据结构

### 2.4.1. VB 内存池配置结构体 (VB\_POOL\_CONFIG\_S)

```
from es_py.common import VB_POOL_CONFIG_S, SYS_CACHE_MODE_E

# VB 内存池配置 (实际的 pybind11 绑定类)
# 属性:
# blkSize: int - 内存块大小 (字节)
# blkCnt: int - 内存块数量
# enRemapMode: SYS_CACHE_MODE_E - 内核态缓存模式
# mmzName: str - MMZ 分区名称

# 使用示例
def create_1080p_pool_config():
    """创建 1080p 视频的 VB 池配置"""
    pool_config = VB_POOL_CONFIG_S()
    pool_config.blkSize = 1920 * 1080 * 3 // 2 # YUV420 格式大小
    pool_config.blkCnt = 16 # 16 个缓存块
    pool_config.enRemapMode = SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED
    pool_config.mmzName = "mmz_nid_0_part_0" # 使用 die0 分区 0
    return pool_config
```

### 2.4.2. VB 系统配置结构体 (VB\_CONFIG\_S)

```
from es_py.common import VB_CONFIG_S, VB_POOL_CONFIG_S, SYS_CACHE_MODE_E

# VB 系统配置 (支持多个内存池)
# 注意: 这是实际的类定义, 由 pybind11 从 C 结构体绑定
# 属性:
# poolCnt: int - 内存池数量
# poolCfgs: CArrayProxy - 内存池配置数组 (支持索引访问)

# 多池配置示例
def setup_multi_pool_config():
    """设置多分辨率视频的 VB 配置"""
    vb_config = VB_CONFIG_S()
    vb_config.poolCnt = 3 # 设置内存池数量

    # 1080p 池 - 通过数组索引直接设置
    pool_1080p = VB_POOL_CONFIG_S()
    pool_1080p.blkSize = 1920 * 1080 * 3 // 2
    pool_1080p.blkCnt = 10
```

```

pool_1080p.enRemapMode = SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED
pool_1080p.mmzName = "mmz_nid_0_part_0"
vb_config.poolCfgs[0] = pool_1080p # 使用索引访问

# 720p 池
pool_720p = VB_POOL_CONFIG_S()
pool_720p.blkSize = 1280 * 720 * 3 // 2
pool_720p.blkCnt = 12
pool_720p.enRemapMode = SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED
pool_720p.mmzName = "mmz_nid_0_part_0"
vb_config.poolCfgs[1] = pool_720p

# 4K 池 (使用 die1)
pool_4k = VB_POOL_CONFIG_S()
pool_4k.blkSize = 3840 * 2160 * 3 // 2
pool_4k.blkCnt = 6
pool_4k.enRemapMode = SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED
pool_4k.mmzName = "mmz_nid_1_part_0" # 使用 die1 的分区
vb_config.poolCfgs[2] = pool_4k

# 读取配置示例
first_pool = vb_config.poolCfgs[0] # 获取第一个池配置
print(f"第一个池大小: {first_pool.blkSize}")

return vb_config

# poolCfgs 是一个 CArrayProxy 对象, 支持:
# - 索引访问: config.poolCfgs[i]
# - 长度获取: len(config.poolCfgs)
# - 迭代: for pool in config.poolCfgs
# - 切片: config.poolCfgs[0:2]

```

### 提示

`VB_CONFIG_S` 是通过 `pybind11` 从 C 结构体自动生成的 Python 类。`poolCfgs` 属性返回一个 `CArrayProxy` 对象, 它提供类似 Python 列表的接口 (索引、迭代等), 但直接操作底层 C 数组。

### 2.4.3. 通道结构体 (CHN\_S)

```

from es_py.common import CHN_S, MOD_ID_E

# 通道标识结构体 (实际的 pybind11 绑定类)
# 属性:
# modId: int - 模块 ID (MOD_ID_E 枚举值)
# nId: int - Node ID (节点 ID)
# devId: int - 设备 ID
# chnId: int - 通道 ID

# 通道创建和绑定示例
def create_video_pipeline():
    """创建视频处理管线: VDEC -> VPS -> VO"""

    # 解码器通道
    vdec_chn = CHN_S()
    vdec_chn.modId = MOD_ID_E.ES_ID_VDEC
    vdec_chn.devId = 0
    vdec_chn.chnId = 0

```

```

# VPS 组和通道
vps_grp_chn = CHN_S()
vps_grp_chn.modId = MOD_ID_E.ES_ID_VPS
vps_grp_chn.devId = 0 # VPS 组 ID
vps_grp_chn.chnId = 0 # VPS 通道 ID

# 视频输出通道
vo_chn = CHN_S()
vo_chn.modId = MOD_ID_E.ES_ID_VO
vo_chn.devId = 0 # VO 设备 ID
vo_chn.chnId = 0 # VO 通道 ID

# 绑定管线
ret1 = es_py.sys.bind(vdec_chn, vps_grp_chn) # VDEC->VPS
ret2 = es_py.sys.bind(vps_grp_chn, vo_chn) # VPS->VO

return ret1 == 0 and ret2 == 0

```

#### 2.4.4. 设备缓冲区结构体 (ES\_DEV\_BUF\_S)

```

from es_py.common import ES_DEV_BUF_S, ES_TENSOR_S, SYS_CACHE_MODE_E
from es_py import memory

# 设备缓冲区结构体（实际的 pybind11 绑定类）
# 属性：
# memFd: int - 内存文件描述符
# offset: int - 偏移量
# size: int - 缓冲区大小
# reserve: int - 保留字段

# 使用场景 1: 使用已有的 NPU 缓冲区 (fd 已分配)
def create_dev_buf_from_npu_buffer(npu_buf):
    """从 NPU 缓冲区创建设备缓冲区（引用现有 fd）"""
    pData = ES_DEV_BUF_S()
    pData.memFd = int(npu_buf.fd) # 使用 NPU 分配的文件描述符
    pData.size = int(npu_buf.size) # 缓冲区大小
    pData.offset = 0 # 通常从偏移 0 开始
    return pData

# 使用场景 2: 分配新内存并创建设备缓冲区
def allocate_and_create_dev_buf(data_size: int, device_id: int = 0):
    """分配内存并创建设备缓冲区（新分配 fd）"""
    # 1. 分配内存
    mmz_name = f"mmz_nid_{device_id}_part_0"
    fd = memory.memory_alloc(
        SYS_CACHE_MODE_E.SYS_CACHE_MODE_NOCACHE,
        f"tensor_buffer_{data_size}",
        mmz_name,
        data_size
    )

    if fd <= 0:
        raise RuntimeError(f"内存分配失败: size={data_size}")

    # 2. 创建设备缓冲区
    pData = ES_DEV_BUF_S()
    pData.memFd = int(fd) # 新分配的文件描述符
    pData.size = int(data_size) # 缓冲区大小

```

```

pData.offset = 0          # 偏移量

# 3. 可选: 写入数据
memview, p_vir_addr = memory.memory_map(fd, data_size,
↪  SYS_CACHE_MODE_E.SYS_CACHE_MODE_NOCACHE)
# memview[:data_size] = your_data # 写入数据
memory.memory_unmap(p_vir_addr, data_size)

return pData

# 使用场景 3: 将 ES_DEV_BUF_S 关联到张量
def attach_buffer_to_tensor(tensor: ES_TENSOR_S, data_size: int):
    """为张量分配并关联设备缓冲区"""
    # 分配内存
    fd = memory.memory_alloc(
        SYS_CACHE_MODE_E.SYS_CACHE_MODE_NOCACHE,
        f"detection_out_{id(tensor)}",
        "mmz_nid_0_part_0",
        data_size
    )

    if fd <= 0:
        raise RuntimeError("张量内存分配失败")

    # 创建缓冲区并关联到张量
    pData = ES_DEV_BUF_S()
    pData.memFd = fd
    pData.offset = 0
    pData.size = data_size
    tensor.set_pData(pData) # 将缓冲区关联到张量

    return tensor

```

#### 2.4.5. NPU 张量相关结构体

```

from es_py.npu import NPU_TENSOR_S, NPU_DIMS4_S, NPU_DATA_TYPE_E

# NPU_DIMS4_S (实际的 pybind11 绑定类)
# 属性:
# n: int - batch 维度
# c: int - channel 维度
# h: int - height 维度
# w: int - width 维度

# NPU_TENSOR_S (实际的 pybind11 绑定类)
# 属性:
# name: str - 张量名称
# bufferSize: int - 缓冲区大小
# dims: NPU_DIMS4_S - 张量维度
# dataFormat: int - 数据格式
# dataType: NPU_DATA_TYPE_E - 数据类型

# 创建 NPU 张量示例
def create_npu_tensor():
    tensor = NPU_TENSOR_S()
    tensor.name = "input_tensor"
    tensor.dataType = NPU_DATA_TYPE_E.NPU_DATA_TYPE_FP32

    # 设置维度 (1x3x224x224)

```

```

dims = NPU_DIMS4_S()
dims.n, dims.c = 1, 3
dims.h, dims.w = 224, 224
tensor.dims = dims

return tensor

```

#### 2.4.6. 视频帧结构体 (VIDEO\_FRAME\_S)

```

from es_py.common import VIDEO_FRAME_S, PIXEL_FORMAT_E

# 视频原始图像帧结构体 (实际的 pybind11 绑定类)
# **类型归属**: [公共] 公共类型 (es_py.common)
# **使用模块**: VPS, VO, VDEC, VENC
# 属性:
# width: int - 图像宽度
# height: int - 图像高度
# field: int - 帧场模式 (暂未使用)
# pixelFormat: PIXEL_FORMAT_E - 视频图像像素格式
# dynamicRange: int - 动态范围 (暂未使用)
# colorGamut: int - 色域范围 (暂未使用)
# stride: list[int] - 图像数据跨距 (3 个元素数组)
# offset: list[int] - 图像数据偏移 (3 个元素数组)
# fd: int - 数据 buffer 文件描述符
# maxLuminance: int - 显示图像的最大亮度 (暂未使用)
# minLuminance: int - 显示图像的最小亮度 (暂未使用)
# PTS: int - 图像时间戳 (微秒)
# privateData: int - 私有数据 (暂未使用)

# 使用示例
def create_video_frame():
    """创建 1080p NV12 格式的视频帧"""
    frame = VIDEO_FRAME_S()
    frame.width = 1920
    frame.height = 1080
    frame.pixelFormat = PIXEL_FORMAT_E.PIXEL_FORMAT_NV12
    frame.PTS = 0 # 时间戳
    return frame

```

#### 2.4.7. 视频帧信息结构体 (VIDEO\_FRAME\_INFO\_S)

```

from es_py.common import VIDEO_FRAME_INFO_S, VIDEO_FRAME_S, MOD_ID_E

# 视频图像帧信息结构体 (实际的 pybind11 绑定类)
# **类型归属**: [公共] 公共类型 (es_py.common)
# **使用模块**: VO, VPS, VDEC
# 属性:
# videoFrame: VIDEO_FRAME_S - 视频图像帧
# poolId: int - 视频缓存池 ID
# modId: MOD_ID_E - 当前帧数据由哪个硬件模块写出

# 使用示例
def create_frame_info():
    """创建视频帧信息"""
    frame_info = VIDEO_FRAME_INFO_S()
    frame_info.videoFrame = VIDEO_FRAME_S()
    frame_info.videoFrame.width = 1920

```

```
frame_info.videoFrame.height = 1080
frame_info.modId = MOD_ID_E.ES_ID_VDEC # 来自 VDEC 模块
return frame_info
```

## 2.5. 枚举类型 API

### 提示

以下枚举类型通过 pybind11 的 `py::enum_` 绑定，可直接作为枚举值使用。

### 2.5.1. 错误级别枚举 (ERR\_LEVEL\_E)

```
from es_py.common import ERR_LEVEL_E

# ERR_LEVEL_E 是 pybind11 枚举类型，可用的值：
# ERR_LEVEL_E.DEBUG = 0    - 调试级别
# ERR_LEVEL_E.INFO = 1    - 信息级别
# ERR_LEVEL_E.WARNING = 2 - 警告级别
# ERR_LEVEL_E.ERROR = 3   - 错误级别
# ERR_LEVEL_E.BUTT = 4    - 边界值

# 使用示例
log_level = ERR_LEVEL_E.ERROR
print(log_level) # 输出: <ERR_LEVEL_E.ERROR: 3>
```

### 2.5.2. 错误码枚举 (EN\_ERR\_CODE\_E)

```
from es_py.common import EN_ERR_CODE_E

# EN_ERR_CODE_E 是 pybind11 枚举类型，常用值：
# EN_ERR_CODE_E.INVALID_DEVID - 无效设备 ID
# EN_ERR_CODE_E.INVALID_CHNID - 无效通道 ID
# EN_ERR_CODE_E.ILLEGAL_PARAM - 非法参数
# EN_ERR_CODE_E.EXIST - 资源已存在
# EN_ERR_CODE_E.UNEXIST - 资源不存在
# EN_ERR_CODE_E.NULL_PTR - 空指针
# EN_ERR_CODE_E.NOT_CONFIG - 未配置
# EN_ERR_CODE_E.NOT_SUPPORT - 不支持
# EN_ERR_CODE_E.NOT_PERM - 无权限
EN_ERR_CODE_E.NOMEM = 0x49    # 内存不足
EN_ERR_CODE_E.NOBUF = 0x4A    # 缓冲区不足
EN_ERR_CODE_E.BUF_EMPTY = 0x4B # 缓冲区为空
EN_ERR_CODE_E.BUF_FULL = 0x4C  # 缓冲区已满
EN_ERR_CODE_E.SYS_NOTREADY = 0x4D # 系统未就绪
EN_ERR_CODE_E.BUSY = 0x4E     # 系统忙
```

### 2.5.3. 载荷类型枚举 (PAYLOAD\_TYPE\_E)

```
from es_py.common import PAYLOAD_TYPE_E

# 视频编码格式
PAYLOAD_TYPE_E.PT_H264 = 96    # H.264 编码
PAYLOAD_TYPE_E.PT_H265 = 265   # H.265/HEVC 编码
```

```
PAYLOAD_TYPE_E.PT_MJPEG = 1002    # Motion JPEG 编码
PAYLOAD_TYPE_E.PT_JPEG = 26       # JPEG 编码

# 音频编码格式
PAYLOAD_TYPE_E.PT_G711A = 8       # G.711 A-law
PAYLOAD_TYPE_E.PT_G711U = 0       # G.711  $\mu$ -law
PAYLOAD_TYPE_E.PT_AAC = 37        # AAC 编码
PAYLOAD_TYPE_E.PT_MP3 = 43        # MP3 编码
```

#### 2.5.4. 视频相关枚举 (像素格式)

```
from es_py.common import PIXEL_FORMAT_E

# YUV 格式
PIXEL_FORMAT_E.PIXEL_FORMAT_YUV400 = 0    # 灰度
PIXEL_FORMAT_E.PIXEL_FORMAT_YV12 = 1      # YV12 格式
PIXEL_FORMAT_E.PIXEL_FORMAT_I420 = 2      # I420 格式
PIXEL_FORMAT_E.PIXEL_FORMAT_YUV444P = 3    # YUV444 格式
PIXEL_FORMAT_E.PIXEL_FORMAT_NV12 = 4      # NV12 格式
PIXEL_FORMAT_E.PIXEL_FORMAT_NV21 = 5      # NV21 格式
PIXEL_FORMAT_E.PIXEL_FORMAT_NV16 = 6      # NV16 格式
PIXEL_FORMAT_E.PIXEL_FORMAT_NV61 = 7      # NV61 格式

# RGB 格式
PIXEL_FORMAT_E.PIXEL_FORMAT_RGB565 = 8     # RGB565 (16 位)
PIXEL_FORMAT_E.PIXEL_FORMAT_BGR565 = 9     # BGR565 (16 位)
PIXEL_FORMAT_E.PIXEL_FORMAT_RGB888 = 10    # RGB888 (24 位)
PIXEL_FORMAT_E.PIXEL_FORMAT_BGR888 = 11    # BGR888 (24 位)
PIXEL_FORMAT_E.PIXEL_FORMAT_ARGB8888 = 12  # ARGB8888 (32 位)
PIXEL_FORMAT_E.PIXEL_FORMAT_ABGR8888 = 13  # ABGR8888 (32 位)
PIXEL_FORMAT_E.PIXEL_FORMAT_RGBA8888 = 14  # RGBA8888 (32 位)
PIXEL_FORMAT_E.PIXEL_FORMAT_BGRA8888 = 15  # BGRA8888 (32 位)
```

#### 2.5.5. 操作模式和控制枚举

```
from es_py.common import OPERATION_MODE_E, ROTATION_E

# 操作模式
OPERATION_MODE_E.OPERATION_MODE_AUTO = 0   # 自动模式
OPERATION_MODE_E.OPERATION_MODE_MANUAL = 1  # 手动模式

# 旋转角度
ROTATION_E.ROTATION_0 = 0                   # 不旋转
ROTATION_E.ROTATION_90 = 1                  # 顺时针 90 度
ROTATION_E.ROTATION_180 = 2                 # 顺时针 180 度
ROTATION_E.ROTATION_270 = 3                 # 顺时针 270 度
ROTATION_E.ROTATION_FLIP_X = 4              # 水平翻转
ROTATION_E.ROTATION_FLIP_Y = 5              # 垂直翻转
```

#### 2.5.6. VB 源类型枚举 (VB\_SOURCE\_E)

```
from es_py.common import VB_SOURCE_E

# VB 内存来源
VB_SOURCE_E.VB_SOURCE_COMMON = 0           # 公共内存池
```

```
VB_SOURCE_E.VB_SOURCE_MODULE = 1  # 模块私有内存池
VB_SOURCE_E.VB_SOURCE_PRIVATE = 2  # 私有内存
VB_SOURCE_E.VB_SOURCE_USER = 3    # 用户分配内存
```

### 2.5.7. 系统配置相关结构体

```
from es_py.common import SYS_VERSION_S, BIND_DEST_S, CHN_S, MOD_ID_E

# 系统版本信息（实际的 pybind11 绑定类）
# 属性：
# version: str (可读写) - 版本字符串

# 绑定目标结构体（实际的 pybind11 绑定类）
# 属性：
# num: int - 目标通道数量
# chn: CArrayProxy - 通道数组（CHN_S 类型，通过索引访问）

# 重要说明：chn 数组成员的访问方式
# BIND_DEST_S.chn 使用 pybind11 的 FixedCArrayProxy 包装，提供类似 Python 列表的索引访问
# 正确用法：bind_dest.chn[index] = chn_obj（索引赋值）
# 错误用法：bind_dest.set_chn(index, chn_obj)（这个方法不存在！）

# 使用示例
def create_bind_destination():
    """创建绑定目标配置"""
    bind_dest = BIND_DEST_S()
    bind_dest.num = 2

    # 使用索引访问数组成员
    chn0 = CHN_S()
    chn0.modId = MOD_ID_E.ES_ID_VDEC
    chn0.devId = 0
    chn0.chnId = 0
    bind_dest.chn[0] = chn0 #✓ 正确：使用索引赋值

    chn1 = CHN_S()
    chn1.modId = MOD_ID_E.ES_ID_VPS
    chn1.devId = 0
    chn1.chnId = 1
    bind_dest.chn[1] = chn1 #✓ 正确：使用索引赋值

    return bind_dest
```

## 2.6. 工具函数和编程方法

### 2.6.1. 内存模块函数（memory）

```
from es_py import memory

# 基础内存管理
mem_fd = memory.memory_alloc(cache_type, p_str_mmb, str_zone, length)
mem_fd2 = memory.malloc_to_memory_fd(orig_ptr, length, cache_type)
memory.memory_flush_cache(mem_fd)
memory.memory_flush_cache_all_by_nid(nid)
memory.memory_flush_cache_all()
memory.memory_free(mem_fd)
```

```
# 映射与反映射 (memory_map 返回 memoryview + 虚拟地址 capsule)
memview, p_vir_addr = memory.memory_map(mem_fd, size, cache_type)
memory.memory_unmap(p_vir_addr, size)

# mem_fd 处理
sub_mem_fd = memory.memory_split_fd(mem_fd, offset, length)
recv_mem_fd = memory.memory_retrieve_fd(sub_mem_fd)
inner_fd = memory.get_internal_fd(mem_fd)
mem_fd3 = memory.get_internal_memory_fd(inner_fd)
mem_size = memory.get_memory_size(mem_fd)

# VB 相关
pool_id = memory.video_buffer_create_pool(pool_cfg)
memory.video_buffer_destroy_pool(pool_id)
blk_mem_fd = memory.video_buffer_get_block(pool_id, blk_size, str_zone="")
memory.video_buffer_release_block(blk_mem_fd)

iova_capsule = memory.video_buffer_alloc_iova(blk_mem_fd, vb_uid)
blk_mem_fd2 = memory.video_buffer_iova2_handle(vb_uid, iova_capsule)
iova_int = memory.iova_capsule_to_int(iova_capsule)
memory.video_buffer_free_iova(vb_uid, blk_mem_fd)
```

### 提示

py\_memory.cpp 中导出的函数名均为 snake\_case 风格，文档示例已与绑定名称保持一致。调用失败时会抛出 ESMemoryError（无需手动检查返回码）。

## 2.6.2. 内存对齐工具函数

```
from es_py import common

# 向上对齐到指定边界
aligned_up = common.ES_ALIGN_UP(1000, 64) # 对齐到 64 字节边界 -> 1024
aligned_up = common.ES_ALIGN_UP(1920, 16) # 对齐到 16 字节边界 -> 1920

# 向下对齐到指定边界 (a 必须是 2 的幂)
aligned_down = common.ES_ALIGN_DOWN(1000, 64) # 向下对齐 -> 960

# 向上除法 (除法结果向上取整)
div_result = common.ES_DIV_UP(1920, 16) # 1920/16 向上取整 -> 120
```

## 2.6.3. 图像缓冲区大小计算函数

```
from es_py import common
from es_py.common import PIXEL_FORMAT_E, COMPRESS_MODE_E

# 获取图像缓冲区大小 (通用方法)
size = common.COMMON_GetPicBufferSize(
    pixelFormat=PIXEL_FORMAT_E.PIXEL_FORMAT_YUV_SEMIPLANAR_420,
    width=1920,
    height=1080,
    alignWidth=16, # 宽度对齐 (通常 16 或 64)
    alignHeight=16 # 高度对齐 (通常 16)
)
print(f"YUV420 1080p 缓冲区大小: {size} bytes")
```

```

# 获取图像缓冲区详细信息 (stride、offset、plane 数)
size, stride, offset, plane = common.COMMON_GetPicBufInfo(
    pixelFormat=PIXEL_FORMAT_E.PIXEL_FORMAT_YUV_SEMIPLANAR_420,
    width=1920,
    height=1080,
    align=16,
    alignHeight=16
)
print(f"缓冲区大小: {size}, stride: {stride}, offset: {offset}, plane: {plane}")

# VDEC 解码器图像缓冲区大小
vdec_size = common.VDEC_GetPicBufferSize(
    pixelFormat=PIXEL_FORMAT_E.PIXEL_FORMAT_YUV_SEMIPLANAR_420,
    width=1920,
    height=1080,
    align=16
)

# VI 原始图像缓冲区大小 (支持压缩)
vi_size = common.VI_GetRawBufferSize(
    width=1920,
    height=1080,
    pixelFormat=PIXEL_FORMAT_E.PIXEL_FORMAT_YUV_SEMIPLANAR_420,
    cmpMode=COMPRESS_MODE_E.COMPRESS_MODE_NONE,
    align=16
)

```

#### 2.6.4. 颜色处理工具函数

```

from es_py import common

# RGB 颜色组装与拆分
color = common.ES_RGB(255, 128, 64) # 组装 24 位 RGB 颜色
r = common.ES_RGB_R(color)          # 提取红色分量 -> 255
g = common.ES_RGB_G(color)          # 提取绿色分量 -> 128
b = common.ES_RGB_B(color)          # 提取蓝色分量 -> 64

# YUV 颜色组装与拆分 (10-bit)
yuv_color = common.ES_YUV(512, 512, 512) # 组装 30 位 YUV 颜色
y = common.ES_YUV_Y(yuv_color)          # 提取 Y 分量
u = common.ES_YUV_U(yuv_color)          # 提取 U 分量
v = common.ES_YUV_V(yuv_color)          # 提取 V 分量

# YUV 颜色组装与拆分 (8-bit)
yuv8_color = common.ES_YUV_8BIT(128, 128, 128)
y8 = common.ES_YUV_8BIT_Y(yuv8_color)
u8 = common.ES_YUV_8BIT_U(yuv8_color)
v8 = common.ES_YUV_8BIT_V(yuv8_color)

```

#### 2.6.5. 字节序和分数工具函数

```

from es_py import common

# 字节序转换
big_endian = common.ES_ENDIAN32(0x12345678) # 32 位大小端转换
little_endian = common.ES_ENDIAN16(0x1234)  # 16 位大小端转换

```

```
# 检查系统字节序
is_little = common.ES_IS_LITTLE_END()    # 返回 True/False

# 分数组装与拆分
fraction = common.ES_FRACTION32(30, 1)    # 组装分数 1/30
numerator = common.ES_NUMERATOR32(fraction) # 提取分子 -> 1
denominator = common.ES_DENOMINATOR32(fraction) # 提取分母 -> 30
```

### 2.6.6. 实用编程示例

```
from es_py import common
from es_py.common import PIXEL_FORMAT_E

def calculate_yuv420_buffer_size(width, height):
    """计算 YUV420 格式的内存大小"""
    return common.COMMON_GetPicBufferSize(
        PIXEL_FORMAT_E.PIXEL_FORMAT_YUV_SEMIPLANAR_420,
        width, height, 16, 16
    )

def align_to_page_size(size):
    """对齐到 4K 页面边界"""
    return common.ES_ALIGN_UP(size, 4096)

def create_aligned_buffer(width, height):
    """创建对齐的缓冲区"""
    # 计算缓冲区大小
    buffer_size = calculate_yuv420_buffer_size(width, height)
    # 对齐到页面边界
    aligned_size = align_to_page_size(buffer_size)
    return aligned_size

# 使用示例
size = create_aligned_buffer(1920, 1080)
print(f"1080p YUV420 对齐缓冲区大小: {size} bytes")
```

### 3. Sample 示例

ESWIN SDK 提供了 Python 示例程序集，用于演示 ESWIN EIC77 芯片平台的 API 使用方法。这些示例通过可运行的代码展示硬件加速功能的调用方式。

从 v1.1 起，示例源码和运行数据由 `pyessdk-eswin-sample` wheel 独立提供；`es_py` wheel 仅提供核心 Python 绑定。运行本章示例前，请同时安装 `es_py` 和 `pyessdk-eswin-sample`。

#### 3.1. 功能模块

主要功能模块：硬件加速 + 零拷贝传输 + AI 推理 + 视频处理

#### 3.2. 软件包结构

wheel 包内容说明：

- **es\_py 核心库**：Python 绑定的 ESWIN SDK C++库，由 `es_py` wheel 提供
- **示例程序**：`memory_sample`、`memcp_sample`、`npu_runtime_sample`、`video_sample`，由 `pyessdk-eswin-sample` wheel 提供
- **测试数据**：包含 YOLOv3/YOLOv11n 模型和测试图片，由 `pyessdk-eswin-sample` wheel 提供

#### 3.3. 示例模块概览

| 模块名称                            | 功能描述                           | 应用说明                   |
|---------------------------------|--------------------------------|------------------------|
| <code>npu_runtime_sample</code> | VDEC 解码+VPS 预处理+NPU 推理+DSP 后处理 | 端到端 AI 推理流水线，多硬件模块协同工作 |
| <code>memory_sample</code>      | 内存管理示例                         | 基础内存操作                 |
| <code>memcp_sample</code>       | DMA 内存复制示例                     | 高速数据传输                 |
| <code>video_sample</code>       | 视频处理示例集                        | 单一模块视频处理               |

#### 3.4. 示例分类

| 分类    | 数量   | 功能描述                                        |
|-------|------|---------------------------------------------|
| 多模块联动 | 1 个  | VDEC+VPS+NPU+DSP 流水线，YOLO 系列检测、多流并发、同步/异步推理 |
| 内存管理  | 2 个  | 通用内存、VB 内存池、进程间传输                           |
| 内存复制  | 1 个  | DMA 异步内存复制                                  |
| 视频处理  | 11 个 | 编解码、显示、图像处理、流水线                             |

#### 3.5. 内存管理示例

##### 3.5.1. memory\_test.py - 内存测试套件

功能描述：

`memory_test.py` 是 ESWIN SDK 中的内存管理测试示例，提供内存子系统功能验证，涵盖从基础内存分配到高级 VB 内存池管理的全套测试用例。

主要功能：

- 通用内存管理：支持 system、CMA、MMZ 等多种内存区域
- VB 内存池系统：Video Buffer 内存池的创建和管理
- 模块内存管理：针对 VENC、VDEC 等特定模块的内存池
- IOVA 地址映射：硬件设备间的地址映射和转换
- 私有内存池：用户自定义内存池的创建和销毁
- 内存分割功能：大块内存的动态分割管理
- 跨进程传输：线程和进程间的内存文件描述符传输
- 错误处理验证：异常情况下的资源清理和错误恢复

#### ESWIN 芯片硬件特性：

- 统一内存架构：支持 system、CMA、MMZ 多种内存区域统一管理
- VB 内存池：硬件加速模块间的零拷贝内存共享机制
- IOVA 映射：硬件设备虚拟地址空间的高效映射
- 缓存一致性：支持 cached/uncached 模式的缓存管理
- 多模块协同：VENC、VDEC、VPS 等模块的内存资源协调

#### 测试覆盖范围：

- 3 种内存区域：system、linux、cma、mmz\_nid\_0\_part\_0
- 2 种缓存模式：cached 和 uncached 内存映射
- 8 个测试项：覆盖内存管理的各个方面
- 多种内存大小：从 4KB 到 4MB 的不同规格测试
- 并发安全性：多线程和多进程环境下的内存操作

#### 运行方法：

命令格式：

```
python3 -m pyessdk_eswin_sample.memory_sample.memory_test [测试编号]
```

参数说明：

测试编号 1-8 的数字，指定运行单个测试项；0 或无参数运行所有测试

执行示例：

```
# 运行所有测试项
python3 -m pyessdk_eswin_sample.memory_sample.memory_test

# 运行指定测试项
python3 -m pyessdk_eswin_sample.memory_sample.memory_test 1 # 通用内存测试
python3 -m pyessdk_eswin_sample.memory_sample.memory_test 2 # VB 内存测试
python3 -m pyessdk_eswin_sample.memory_sample.memory_test 3 # VB 模块内存测试
python3 -m pyessdk_eswin_sample.memory_sample.memory_test 4 # VB 模块 IOVA 测试
python3 -m pyessdk_eswin_sample.memory_sample.memory_test 5 # 私有池测试
python3 -m pyessdk_eswin_sample.memory_sample.memory_test 6 # VB 内存分割测试
python3 -m pyessdk_eswin_sample.memory_sample.memory_test 7 # 线程间传输测试
python3 -m pyessdk_eswin_sample.memory_sample.memory_test 8 # 错误处理测试
```

### 3.5.2. memory\_test2.py - 进程间内存传输测试

功能描述：

memory\_test2.py 演示进程间内存文件描述符传输机制，展示如何在父子进程间安全地共享内存资源。

主要功能：

- 进程间内存文件描述符传输
- 子进程内存接收和操作
- 跨进程内存共享机制

**运行方法：**

```
python3 -m pyessdk_eswin_sample.memory_sample.memory_test2
```

**提示**

该示例无命令行参数。

## 3.6. 内存复制示例

### 3.6.1. memcp\_test.py - DMA 内存复制演示

**功能描述：**

memcp\_test.py 是 ESWIN SDK 中的 DMA（Direct Memory Access）内存复制示例，展示硬件加速 DMA 传输 API 的使用方法。

**主要功能：**

- DMA 硬件加速：利用专用 DMA 控制器实现 CPU 零负载的内存传输
- 异步命令队列：支持多个 DMA 操作的队列化管理和批处理
- 大块内存传输：高效处理 2MB 级别的大容量数据传输
- 并发传输管理：支持多路 DMA 通道的并行操作
- 超时控制机制：可配置的传输超时保护和错误恢复
- 内存一致性保证：确保 DMA 传输前后的缓存一致性

**ESWIN 芯片硬件特性：**

- 专用 DMA 引擎：独立的 DMA 控制器，不占用 CPU 资源
- 多通道支持：支持多路并发 DMA 传输操作
- 零拷贝架构：硬件直接访问内存，无需 CPU 参与数据搬移
- 缓存管理：自动处理 cached/uncached 内存的缓存一致性
- 高吞吐量：支持 GB/s 级别的内存传输带宽

**性能特性：**

- 传输速度：2MB 数据传输通常在毫秒级完成
- CPU 零负载：DMA 传输期间 CPU 可执行其他任务
- 高并发：支持多个 DMA 操作同时进行
- 精确控制：5 秒超时控制，确保传输可靠性

**运行方法：**

```
python3 -m pyessdk_eswin_sample.memcp_sample.memcp_test
```

**提示**

该示例无命令行参数。

## 3.7. NPU 推理示例

**多模块硬件联动示例**本示例展示了 VDEC、VPS、NPU、DSP 四大硬件模块的 API 组合调用方法。

### 3.7.1. npu\_runtime\_sample - YOLO 多流检测（多模块联动）

**示例定位：**端到端 AI 推理流水线 API 演示

本示例演示了多个硬件模块的 API 协同调用：

- VDEC 硬件解码 - VPS 硬件预处理 - NPU 硬件推理 - DSP 硬件后处理
- 完整的零拷贝数据流
- 多流并发处理架构
- 同步/异步推理模式

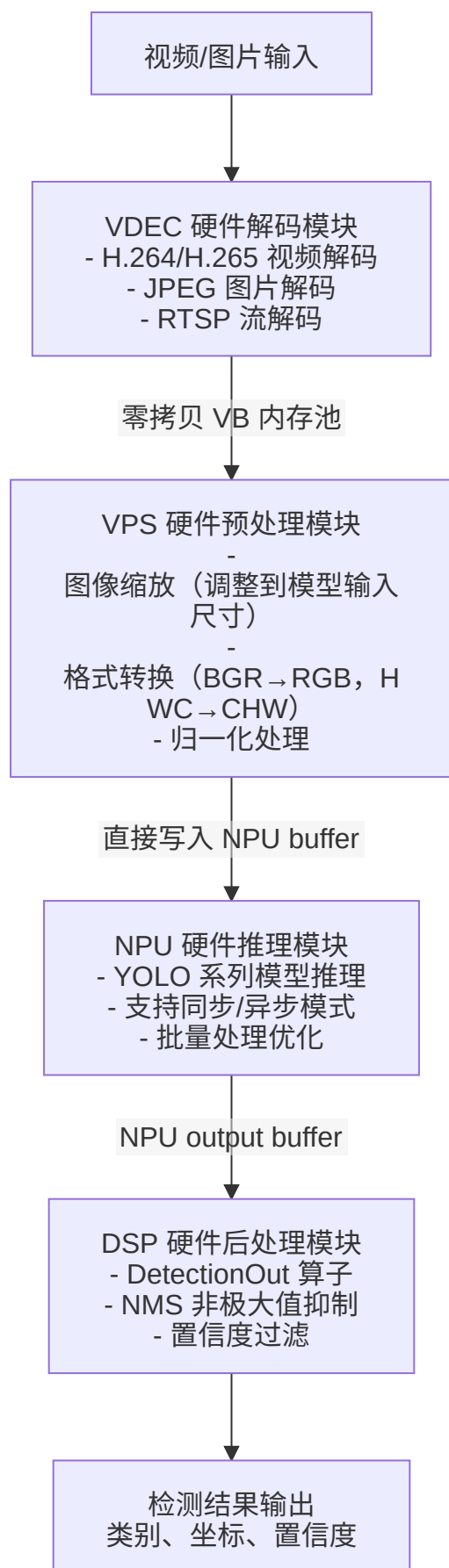
**示例特点：**

- **多模块联动**：展示 4 个硬件模块如何协同工作
- **完整流水线**：从视频输入到检测结果的端到端处理
- **API 综合运用**：演示多个 SDK 模块的组合使用方法
- **典型应用场景**：覆盖常见的 AI 视觉应用需求

用户目标：学习多模块协同的 API 调用方法，了解端到端应用开发流程

**多模块协同架构：**

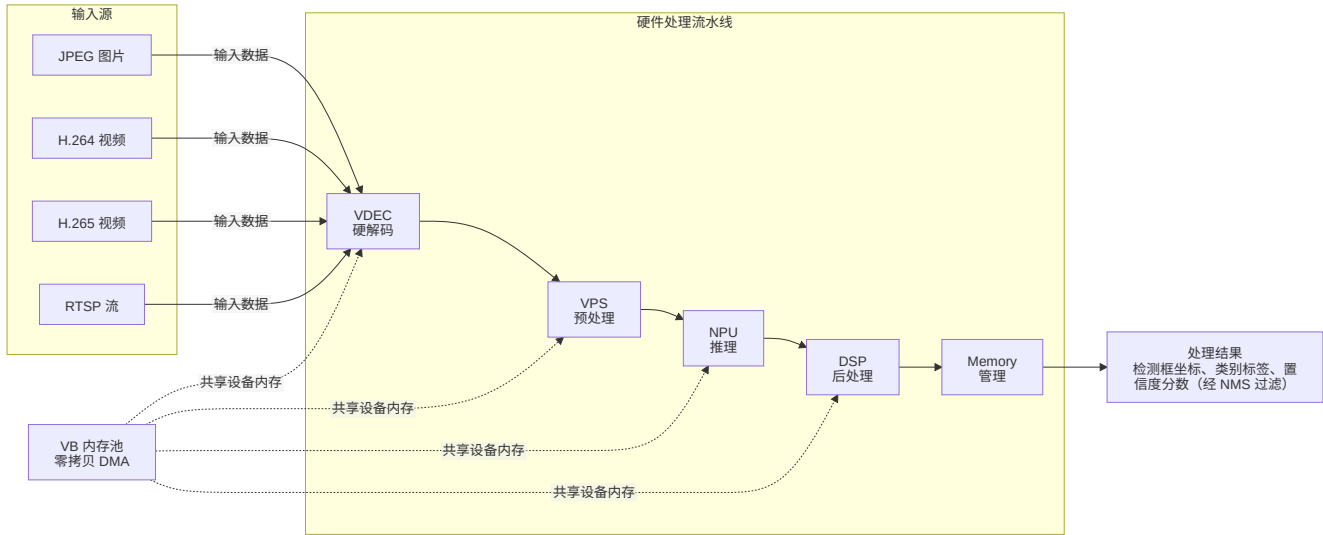
本示例集成四个硬件加速模块，通过 Python SDK 调用模块 API：



主要功能：

1. 多硬件模块联动架构：

本示例展示了端到端 AI 推理流水线的完整 API 调用流程，从视频/图像输入到检测结果输出：



流水线工作流程：

- **输入阶段：**支持 JPEG 图片、H.264/H.265 视频流、RTSP 网络流等多种输入源
- **VDEC 硬件解码：**将压缩的视频/图像数据解码为原始 YUV 格式
- **VPS 硬件预处理：**图像缩放、格式转换、归一化等预处理操作
- **VB 内存池：**零拷贝数据传输，各模块间高效共享数据
- **NPU 推理：**AI 模型推理，生成检测特征
- **DSP 后处理：**NMS 过滤、坐标解码、置信度筛选
- **结果输出：**检测框坐标、类别标签、置信度分数

API 使用方法：

- **硬件加速流水线：**演示 VDEC、VPS、NPU、DSP 等模块 API 的组合调用
- **零拷贝数据传输：**展示 VB 内存池 API 的使用，实现模块间高效数据传递
- **流水线并行处理：**说明异步 API 的调用方式，提高吞吐量
- **同步/异步模式：**演示两种不同推理模式的 API 调用差异

2. YOLO 目标检测能力：

- 支持多种 YOLO 模型：YOLOv3、YOLOv5、YOLOv8、YOLOv11 等
- 多种输入分辨率：416×416、640×640 等
- 支持多路视频流并发检测
- 高精度检测结果输出（类别、坐标、置信度）

3. 多输入源支持：

- **JPEG 图像：**本地 JPEG 文件 (.jpg、.jpeg)
- **原始 H.264/H.265 视频流：**本地文件 (.h264、.h265、.264、.265、.hevc)
- **RTSP 网络流：**支持 H.264/H.265 编码的实时流
- **混合多路输入：**可同时处理多种不同类型的输入源

4. 双推理模式：

- **同步模式 (Sync)：**逐帧处理，适合单路高精度应用
- **异步模式 (Async)：**多路并发，适合多路实时应用，使用流水线并行处理

5. 各硬件模块详细功能：

- VDEC 硬件解码：支持多路并发 H.264/H.265/JPEG 解码
- VPS 硬件预处理：自动缩放、BGR 转 RGB、HWC 转 CHW、归一化

- NPU 推理加速：专用 AI 处理器，高效神经网络推理
- DSP 后处理加速：硬件 NMS、DetectionOut 算子，过滤重复框

ESWIN 芯片硬件加速模块（多模块联动架构）：

本示例展示了 4 个硬件模块的协同工作方式：

### 1. VDEC（视频解码器）：硬件视频/图像解码模块

- H.264/H.265 原始视频流解码（.h264/.h265/.264/.265/.hevc）
- JPEG 图像格式解码
- RTSP 网络流解码（仅支持 H.264/H.265 编码）
- 多路并发解码
- 解码输出直接到 VB 内存池
- 不支持 MP4/AVI/MKV 等容器格式

### 2. VPS（视频处理系统）：硬件加速图像预处理

- 图像解码和缩放（支持多种分辨率：416×416、640×640 等）
- 颜色空间转换（BGR 转 RGB）
- 数据格式转换（HWC 转 CHW）
- 像素归一化和量化处理
- 从 VB 内存池读取，直接写入 NPU buffer（零拷贝）

### 3. NPU（神经网络处理单元）：专用 AI 推理加速器

- 执行 YOLO 系列神经网络推理
- 支持同步/异步推理模式
- 批量处理优化
- 从专用 buffer 读取输入，输出到专用 buffer

### 4. DSP（数字信号处理器）：硬件后处理加速

- DetectionOut 硬件算子
- NMS（非极大值抑制）硬件实现
- 置信度过滤
- 坐标解码和类别映射
- 从 NPU output buffer 读取数据

### 5. Memory 子系统：统一内存管理

- VB 内存池统一管理
- DMA 零拷贝数据传输
- 跨模块内存共享

### 6. System 管理：硬件资源协调和设备管理

多模块协同的特点：

- 硬件加速：处理任务在专用硬件上完成
- 零拷贝传输：数据在各模块间直接传递
- 流水线并行：各模块可同时处理不同帧
- 延迟较低：硬件处理延迟低于软件实现
- 能效比高：专用硬件功耗低于通用 CPU 处理

支持的输入类型：

- 图片文件：JPEG 格式图片或包含 JPEG 图片的目录
- 视频文件：原始 H.264/H.265 流文件（.h264/.h265/.264/.265/.hevc 格式）
- 网络流：RTSP 实时流（仅支持 H.264/H.265 编码）
- 混合输入：可同时处理多种不同类型的输入源

## 提示

不支持 MP4/AVI/MKV 等容器格式。如需处理容器格式视频，请先使用 FFmpeg 转换为原始流：

```
# 从 MP4 提取 H.264 原始流（推荐，无需重新编码）
ffmpeg -i input.mp4 -c:v copy -bsf:v h264_mp4toannexb output.h264

# 从 MP4 提取 H.265 原始流（推荐，无需重新编码）
ffmpeg -i input.mp4 -c:v copy -bsf:v hevc_mp4toannexb output.h265
```

## 额外依赖说明：

OpenCV（可选）：用于结果可视化（绘制检测框）。如不需要可视化功能，可跳过安装

## 运行方法：

命令格式：

```
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream [选项]
```

## 参数说明：

- `--model`：YOLO 模型文件路径（.model 文件，必需）
- `--input`：输入源列表（支持多个，空格分隔）
  - JPEG 图片文件或包含 JPEG 图片的目录
  - 原始视频流文件（.h264/h265/.264/.265/.hevc）
  - RTSP 网络流（仅支持 H.264/H.265 编码）
  - 可混合使用多种类型
  - 不支持 MP4/AVI/MKV 等容器格式
- `--batch-size`：NPU 批处理大小（默认：1）
- `--inference-mode`：推理模式，sync（同步）或 async（异步，默认）
- `--preprocess-config`：预处理配置文件路径（JSON，可选，默认自动检测）
- `--postprocess-config`：后处理配置文件路径（JSON，可选，默认自动检测）
- `--classes`：类别文件路径（可选，默认自动检测）
- `--output`：输出目录（保存检测结果文本文件）
- `--visualize`：在图片上绘制边框（仅 JPEG 输入，需要 OpenCV）
- `--vis-output`：可视化图片输出目录（默认：./visualized）
- `--device`：NPU 设备 ID（默认：0）
- `--max-frames`：每个流解码的最大帧数（默认：None，解码所有帧）
- `--log-level`：日志级别（DEBUG/INFO/WARNING/ERROR，默认：INFO）

## 执行示例：

```
# 获取 wheel 包中的数据路径（如果使用 wheel 包中的测试数据）
INSTALL_PATH=$(python3 -c "import pyessdk_eswin_sample; print(pyessdk_eswin_sample.__path__[0])")

# 1. 单张 JPEG 图片检测
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream \
  --model "${INSTALL_PATH}/data/models/yolov3/git_yolov3_416_mix_1x3x416x416_dyn_npu_b1.model" \
  --input "${INSTALL_PATH}/data/models/yolov3/input/pictures/input0/bus.jpg"

# 2. JPEG 图片目录批量检测
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream \
  --model "${INSTALL_PATH}/data/models/yolov3/git_yolov3_416_mix_1x3x416x416_dyn_npu_b1.model" \
  --input "${INSTALL_PATH}/data/models/yolov3/input/pictures/input0/" \
```

```
--output ./results
```

# 3. 视频文件检测 (解码前 100 帧)

# 注意: 请替换为实际的模型文件和视频文件路径

```
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream \
--model "${INSTALL_PATH}/data/models/yolov3/git_yolov3_416_mix_1x3x416x416_dyn_npu_b1.model" \
--input your_video.h265 \
--max-frames 100 \
--output ./results
```

# 4. 多路视频并发检测

# 注意: 请准备多个视频文件进行测试

```
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream \
--model "${INSTALL_PATH}/data/models/yolov3/git_yolov3_416_mix_1x3x416x416_dyn_npu_b1.model" \
--input video1.h265 video2.h264 video3.h265 \
--batch-size 2 \
--output ./results
```

# 5. RTSP 实时流检测

# 注意: 请替换为实际的 RTSP 流地址

```
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream \
--model "${INSTALL_PATH}/data/models/yolov3/git_yolov3_416_mix_1x3x416x416_dyn_npu_b1.model" \
--input rtsp://192.168.1.100:554/stream1 rtsp://192.168.1.101:554/stream2
```

# 6. 混合输入源 (图片+视频+RTSP)

# 注意: 可以混合使用不同类型的输入源

```
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream \
--model "${INSTALL_PATH}/data/models/yolov3/git_yolov3_416_mix_1x3x416x416_dyn_npu_b1.model" \
--input "${INSTALL_PATH}/data/models/yolov3/input/pictures/input0/" your_video.h265
↪ rtsp://192.168.1.100:554/stream
```

# 7. 同步推理模式

```
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream \
--model "${INSTALL_PATH}/data/models/yolov3/git_yolov3_416_mix_1x3x416x416_dyn_npu_b1.model" \
--input your_video.h265 \
--inference-mode sync
```

# 8. JPEG 检测结果可视化 (需要 OpenCV)

```
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream \
--model "${INSTALL_PATH}/data/models/yolov3/git_yolov3_416_mix_1x3x416x416_dyn_npu_b1.model" \
--input "${INSTALL_PATH}/data/models/yolov3/input/pictures/input0/" \
--visualize \
--vis-output ./visualized
```

# 9. 查看帮助信息

```
python3 -m pyessdk_eswin_sample.npu_sample.npu_runtime_sample.main_multi_stream --help
```

### 多流处理特性:

- 支持同时处理多路输入流
- 独立线程处理每路输入
- 批量拼接提高 NPU 利用率
- 异步推理提高吞吐量

### 硬件加速特性:

- 硬件流水线: VDEC(视频/图像解码) → VPS 预处理 → NPU 推理 → DetectionOut 后处理
- 零拷贝传输: 模块间通过 VB 内存池实现高效数据传输
- 多流并发处理: 支持多路输入流同时处理
- 批量处理优化: 多帧批量拼接提高 NPU 利用率
- 专用算子加速: DetectionOut、NMS 等后处理算子硬件实现
- 同步/异步模式: 灵活的推理模式适应不同场景需求

**提示**

- 检测成功，能够准确识别目标和置信度
- 如果使用 `-visualize` 但出现 “postprocess failed: No module named ‘cv2’ ”，需要安装 `opencv-python`
- 运行时警告不影响功能正常执行
- 可使用 `-max-frames` 限制解码帧数以加快测试

## 3.8. 视频处理示例

### 3.8.1. py\_vdec - 视频解码示例

**用户目标：**使用 ESWIN 芯片解码各种视频格式

**功能简介：**py\_vdec 是 ESWIN SDK 中的视频解码示例，展示 VDEC 模块 API 的使用方法，支持多种主流视频编码格式和分辨率。

**主要功能：**

- 多编码格式支持：H.264、H.265、JPEG 等主流视频编码的硬件解码
- 多分辨率支持：从 1080P 到 4K 的全分辨率范围解码能力
- 多通道并行解码：支持同时进行多路视频流的并行解码
- VB 内存池管理：高效的视频帧缓存池管理和复用
- 文件 I/O 管理：支持视频流读取和解码帧输出存储
- 异常处理：完善的信号处理和资源清理机制

**ESWIN 芯片硬件特性：**

- VDEC 硬件引擎：专用的视频解码硬件单元，实现高效率解码
- 多核并行处理：支持多组解码器同时工作，最大分 5 组
- 硬件流水线：解码器、VB 内存池、DMA 的紧密协同
- 零拷贝数据流：解码帧直接输出到 VB 内存池，无需中间拷贝

**支持的解码模式：**

- 模式 0: 4K H.264 解码 - 单组双通道输出
- 模式 1: 4K JPEG×4 - 4 组并行，每组双通道
- 模式 2: 1080P H.264×5 - 性能测试模式，5 组高并发
- 模式 3: 1080P H.265×5 - 性能测试模式，5 组高并发
- 模式 4: 混合编码 - H.264+H.265 同时解码
- 模式 5: Die-to-Die - 跨芯片协同解码模式

**性能指标：**

- 4K@60fps: 单路 4K H.264/H.265 解码能力
- 多路并行: 支持多路 1080P 流同时解码
- 低延迟: 硬件解码延迟小于 1 帧时间
- 低功耗: 专用硬件功耗显著低于软件解码

**运行方法：**

执行命令：

```
python3 -m pyessdk_eswin_sample.video_sample.vdec.py_vdec
```

**提示**

该示例无命令行参数，直接运行即可。需要视频解码器硬件支持和 root 权限或 video 组权限。

### 3.8.2. py\_venc - 视频编码示例

#### 功能描述：

py\_venc 是 ESWIN SDK 中的视频编码示例，展示 VENC 模块 API 的使用方法，支持多种主流编码格式和编码特性。

#### 主要功能：

- 多编码格式支持：H.264、H.265、MJPEG、JPEG 等主流编码的硬件实现
- 多种码率控制模式：CBR、VBR、FixQP 等专业码率控制算法
- GOP 结构优化：NORMALP、DUALREF、BIPREDB 等多种 GOP 模式
- QP 映射支持：支持用户自定义的量化参数映射
- 多通道并行编码：支持同时进行多路视频流的并行编码
- Die-to-Die 支持：跨芯片的分布式编码处理能力

#### ESWIN 芯片硬件特性：

- VENC 硬件引擎：专用的视频编码硬件单元，实现低功耗高效编码
- 智能码率控制：硬件级的码率控制算法，实时动态调整
- 高级编码特性：支持 B 帧、参考帧管理等高级编码技术
- 实时性保障：硬件流水线设计，确保实时编码性能

#### 支持的编码模式：

- 模式 0: H.265@4K + H.264@1080P - 混合编码双通道
- 模式 1: H.265@4K + H.264@4K - 高分辨率双通道
- 模式 2: QP 映射模式 - 支持用户自定义量化参数
- 模式 3: MJPEG@4K + JPEG@4K - 静态图像编码
- 模式 4: 多通道@1080P - 高并发性能测试
- 模式 5: Die-to-Die - 4 通道跨芯片协同编码

#### 交互式配置功能：

- GOP 模式选择：支持用户交互选择 NORMALP/DUALREF/BIPREDB
- 码率控制选择：支持用户交互选择 CBR/VBR/FixQP 模式
- 参数实时调整：运行时可动态调整编码参数

#### 性能指标：

- 4K@60fps: 单路 4K H.264/H.265 编码能力
- 多路并行: 支持多路 1080P 流同时编码
- 低延迟: 硬件编码延迟小于 2 帧时间
- 高压缩比: 支持高效的码率控制和质量保证

#### 运行方法：

#### 执行命令：

```
python3 -m pyessdk_eswin_sample.video_sample.venc.py_venc
```

#### 提示

该示例支持交互式配置，运行后可以选择 GOP 模式和码率控制模式。

### 3.8.3. sample\_vo - 视频输出示例

#### 功能描述：

sample\_vo 是 ESWIN SDK 中的视频输出示例，展示 VO 模块 API 的使用方法，支持 HDMI 视频输出到显示设备。

#### 主要功能：

- HDMI 视频输出

- 多种分辨率支持（4K60, 1080P60）
- 多通道同时输出
- 双 die 支持

#### 运行方法：

#### 命令格式：

```
python3 -m pyessdk_eswin_sample.video_sample.vo.sample_vo [index] [die]
```

#### 参数说明：

- index：输出模式
  - 0: HDMI 4K60Hz 模式，支持最多 16 路同时输出
  - 1: HDMI 1080P60Hz 模式，支持最多 16 路同时输出
- die：运行的 die（可选）
  - 0: 仅在 die0 运行
  - 1: 仅在 die1 运行
  - 2: 在 die0 和 die1 同时运行

#### 执行命令：

```
# HDMI 4K60 输出, die0
python3 -m pyessdk_eswin_sample.video_sample.vo.sample_vo 0 0

# HDMI 1080P60 输出, die0
python3 -m pyessdk_eswin_sample.video_sample.vo.sample_vo 1 0

# HDMI 4K60 输出, 双 die
python3 -m pyessdk_eswin_sample.video_sample.vo.sample_vo 0 2
```

#### 提示

不能在桌面环境中运行，需要 HDMI 显示器连接和 root 权限。

### 3.8.4. py\_vps - 视频处理示例

#### 功能描述：

py\_vps 是 ESWIN SDK 中的视频处理系统（Video Processing System）示例，展示 VPS 模块 API 的使用方法。

#### 主要功能：

- 图像缩放：支持从 1080P 到 4K 的双向缩放，基于 HAE 硬件加速
- 图像裁剪：支持任意区域的精确裁剪和提取
- 像素格式转换：NV12、NV16、RGB 等多种格式间的转换
- 图像变换操作：支持旋转、翻转、镜像等几何变换
- 多通道并行处理：同时处理多路视频流的不同变换操作
- 实时流处理：支持连续视频流的实时处理和输出

#### ESWIN 芯片硬件特性：

- VPS 硬件引擎：专用的视频处理硬件单元，集成 HAE 和 VSE 双引擎
- HAE 引擎：Hardware Accelerator Engine，专门处理缩放、翻转等操作
- VSE 引擎：Video Scaling Engine，专门处理多路输出和复杂变换
- 数据流优化：与 VDEC、NPU 等模块的零拷贝数据交换
- 实时性保证：硬件流水线设计，确保实时处理性能

#### 支持的处理模式：

- 模式 0: 1080P->4K HAE 缩放模式

- 模式 1: 4K->1080P HAE 缩放+FLIP 翻转模式
- 模式 2: 1080P VSE 多路输出模式
  - 通道 0: NV12->NV16 4K + HAE FLIP
  - 通道 1: NV12->RGB 1080P
  - 通道 2: NV12 1080P 原始输出
- 模式 3: VSE 多组多路输出模式
- 模式 4: 跨 Die 的 VSE 处理模式

处理能力范围：

- 分辨率支持：支持 128x128 到 4096x4096 的全分辨率范围
- 像素格式：NV12、NV16、NV21、RGB24、ARGB 等主流格式
- 变换操作：0°/90°/180°/270° 旋转，水平/垂直翻转
- 缩放算法：支持多种缩放算法
  - **Bresenham 算法**：速度最快，质量较低（默认）
  - **FIR 滤波器**：5x5 块大小，速度较慢，高质量缩放
  - **双线性 (Bilinear)**：速度和质量平衡
  - **双三次 (Bicubic)**：高质量缩放

性能指标：

- 4K@60fps: 单路 4K 视频流的实时处理能力
- 多路并行: 支持多路 1080P 流同时处理
- 低延迟: 硬件处理延迟极低
- 高吞吐量: 支持 GB/s 级别的数据处理带宽

运行方法：

命令格式：

```
python3 -m pyessdk_eswin_sample.video_sample.vps.py_vps [case_index] [选项]
```

参数说明：

| 简写         | 取值   | 默认 | 作用                                                         |
|------------|------|----|------------------------------------------------------------|
| -w         | 0 1  | 0  | 1=把每路输出写盘<br>/opt/eswin/-<br>data/video/output_*           |
| -r         | ≥ 1  | 1  | 同一 case 循环读帧次数，做<br>压力测试                                   |
| -l         | 0 1  | 0  | 1=从 /opt/eswin/-<br>data/video/input_* 读裸<br>YUV，否则用内置彩条/流 |
| case_index | 0-15 | 必填 | 运行哪个 VPS 示例（见下表）                                           |

快速用例：

| case | 说明                                    |
|------|---------------------------------------|
| 0    | 1080p → 4K 放大                         |
| 1    | 4K → 1080p 缩小+水平翻转                    |
| 2    | 1 入 3 出：4K NV16+翻转、1080p RGB、1080p 原图 |
| 3    | 鱼眼去畸变                                 |
| 4    | 纯格式转换（NV12 ↔ RGB 等）                   |

| case | 说明                          |
|------|-----------------------------|
| 5    | 硬件 OSD 叠加                   |
| 6    | 画矩形                         |
| 7    | 224×224 RGB →归一化 FP32，供 NPU |
| 8    | 画直线                         |
| 9    | 色彩空间转换                      |
| 10   | 任意分辨率缩放                     |
| 11   | 8K 输入→ 4×1080p 输出（四核 HAE）   |
| 12   | Die-1 → Die-2 零拷贝           |
| 13   | Die-2 → Die-1 零拷贝           |
| 14   | 多矩形叠加                       |
| 15   | 硬件 α 混合                     |

常用命令：

```
# 跑一次多通道演示，不存盘
python3 -m pyessdk_eswin_sample.video_sample.vps.py_vps 2

# 保存输出，循环 100 帧，从文件读 YUV
python3 -m pyessdk_eswin_sample.video_sample.vps.py_vps -w 1 -r 100 -l 1 2

# 稳定性测试：case-0 放大 10000 帧
python3 -m pyessdk_eswin_sample.video_sample.vps.py_vps -w 0 -r 10000 0
```

3.9. 复合视频流水线示例

3.9.1. vdec\_venc - 解码+编码流水线

功能描述：

vdec\_venc 是视频转码流水线示例，将输入的视频流解码后直接重新编码为不同的格式或参数。支持 H.264 → H.265、不同码率、不同分辨率等多种转码需求，广泛应用于流媒体和存储优化场景。

运行方法：

执行命令：

```
python3 -m pyessdk_eswin_sample.video_sample.vdec_venc.py_vdec_venc
```

3.9.2. vdec\_vps\_venc - 解码+处理+编码流水线

功能描述：

vdec\_vps\_venc 是视频处理流水线示例，将解码后的视频流经过 VPS 进行缩放、裁剪、格式转换等处理，然后再由 VENC 重新编码输出。适用于 AI 预处理、视频增强、云游戏等需要实时视频处理的场景。

运行方法：

执行命令：

```
python3 -m pyessdk_eswin_sample.video_sample.vdec_vps_venc.py_vdec_vps_venc
```

### 3.9.3. vdec\_vps\_vo - 解码+处理+显示流水线

#### 功能描述：

vdec\_vps\_vo 是视频处理和实时显示的组合流水线示例，将解码后的视频流经过 VPS 进行实时处理（如去噪、增强、特效等），然后直接输出到 HDMI 显示器。适用于实时视频处理应用、数字标牌等场景。

#### 运行方法：

执行命令：

```
python3 -m pyessdk_eswin_sample.video_sample.vdec_vps_vo.py_vdec_vps_vo
```

## 4. 故障排除

本章节汇总了在部署和使用 ESSDK Python SDK 过程中可能遇到的常见问题及其解决方案，旨在帮助开发者快速定位并解决问题。

### 4.1. 安装与环境配置问题

#### 4.1.1. 问题 1: Python 模块导入失败

**【错误现象】** 在执行 Python 脚本时，出现以下任一错误：

```
ImportError: No module named 'es_py'  
ModuleNotFoundError: No module named 'pyessdk_eswin_sample'
```

**【原因分析】**

1. 当前 Python 环境未安装 `es_py`、`pyessdk-eswin-sample` 或其依赖包。
2. Python 虚拟环境 (venv) 未激活，导致使用了错误的 Python 解释器路径。
3. pip 安装源配置错误，未能正确拉取 `es_py` 包。

**【解决方案】** 请按照以下步骤依次排查：

```
# 1. 确认当前使用的 Python 解释器路径  
which python3  
  
# 2. 确认当前 venv 环境中已安装的包  
pip3 list | grep -E "es-py|pyessdk-eswin-sample"  
  
# 3. 强制重新安装 wheel 包（请确保 URL 正确）  
pip3 install --force-reinstall --index-url https://archive.eswincomputing.com/pypi/simple/ es_py  
pip3 install --force-reinstall --index-url https://archive.eswincomputing.com/pypi/simple/  
↳ pyessdk-eswin-sample  
  
# 4. 验证安装结果  
python3 -c "import es_py; print('es_py 安装成功')"
```

#### 4.1.2. 问题 2: 缺少底层系统依赖 (SO 库)

**【错误现象】** 运行程序时报错，提示无法找到共享对象文件：

```
OSError: cannot open shared object file: libessdk_*.so: No such file or directory
```

**【原因分析】** Python 层 (`es_py`) 只是底层 C/C++ SDK 的绑定接口，若系统未安装 ESSDK 的原生 DEB 包，会导致底层 `.so` 库文件缺失。

**【解决方案】** 安装或修复系统级的 SDK 依赖：

```
# 1. 检查 SDK DEB 包的安装状态  
dpkg -l | grep es-sdk  
  
# 2. 更新软件源并安装核心组件  
sudo apt update  
sudo apt install es-sdk-common es-sdk-memory es-sdk-sys  
  
# 3. 更新动态链接库缓存  
sudo ldconfig
```

```
# 4. 验证库链接是否恢复正常
/sbin/ldconfig -p | grep libes_
```

### 4.1.3. 问题 3: Python 版本不兼容

#### 【错误现象】

```
ImportError: Python version mismatch
```

【原因分析】当前运行的 Python 版本与 `es_py` wheel 包编译时所依赖的版本（**Python 3.12.6**）不一致。

#### 【解决方案】

```
# 1. 检查当前 Python 版本
python3 --version
# 预期输出: Python 3.12.6

# 2. 如果版本不匹配, 请使用对应版本的 Python 解释器,
# 或联系 ESWIN 交付工程师获取适配您当前 Python 版本的 wheel 包。
```

## 4.2. 运行时问题

### 4.2.1. 问题 4: 设备权限不足

#### 【错误现象】

```
PermissionError: Operation not permitted
```

【原因分析】访问 `/dev/es_*` 等设备文件需要特定权限, 当前用户不在 `video` 用户组中。

#### 【解决方案】

```
# 方法 1: 临时使用 root 权限运行 (不推荐用于生产环境)
sudo python3 your_script.py

# 方法 2: 永久添加用户到 video 组 (推荐, 需注销后重新登录生效)
sudo usermod -a -G video $USER

# 方法 3: 临时修改设备文件权限 (重启后失效)
ls -la /dev/es_*
sudo chmod 666 /dev/es_*
```

### 4.2.2. 问题 5: 硬件设备不可用

#### 【错误现象】

```
RuntimeError: Device not found
OSError: No such device or address
```

#### 【原因分析】

1. 内核模块未加载。
2. 硬件设备未初始化或未连接。
3. ESWIN 相关驱动未正确安装。

## 【解决方案】

```
# 1. 检查内核模块是否加载
lsmod | grep es_

# 2. 检查设备文件是否存在
ls -la /dev/es_* 2>/dev/null || echo "设备文件不存在"

# 3. 检查 NPU 设备状态
cat /proc/es_npu 2>/dev/null || echo "NPU 设备不可用"

# 4. 检查系统内核日志中是否有相关报错
dmesg | grep -i eswin
dmesg | grep -i es_
```

## 4.3. 调试与诊断工具

## 4.3.1. 综合调试脚本

建议创建一个名为 `debug_eswin.py` 的综合诊断脚本，一键收集系统信息：

```
#!/usr/bin/env python3
"""ESWIN SDK 系统调试脚本"""

import sys
import subprocess
import os

def check_python_env():
    print("=== Python 环境检查 ===")
    print(f"Python 版本: {sys.version}")
    print(f"Python 路径: {sys.executable}")

def check_eswin_modules():
    print("\n=== ESWIN 模块检查 ===")
    try:
        import es_py
        print(f"es_py 导入成功, 位置: {es_py.__file__}")
    except ImportError as e:
        print(f"es_py 导入失败: {e}")
    try:
        import pyessdk_eswin_sample
        print(f"pyessdk_eswin_sample 导入成功")
    except ImportError as e:
        print(f"pyessdk_eswin_sample 导入失败: {e}")

def check_system_deps():
    print("\n=== 系统依赖检查 ===")
    result = subprocess.run(['dpkg', '-l'], capture_output=True, text=True)
    es_packages = [line for line in result.stdout.split('\n') if 'es-sdk' in line]
    if es_packages:
        print(f"已安装 {len(es_packages)} 个 SDK 包")
    else:
        print("警告: 未找到 es-sdk 包")

def check_devices():
    print("\n=== 设备状态检查 ===")
    device_files = subprocess.run(['ls', '/dev/es_*'], capture_output=True, text=True, shell=True)
    if device_files.returncode == 0:
        print("设备文件: " + device_files.stdout.strip())
```

```

else:
    print("警告: 未找到 ESWIN 设备文件")

if __name__ == "__main__":
    check_python_env()
    check_eswin_modules()
    check_system_deps()
    check_devices()
    print("\n=== 调试检查完成 ===")

```

运行方法：

```
python3 debug_eswin.py
```

## 4.4. 常用命令速查表

### 4.4.1. Python 环境检查

```

which python3          # 检查当前 Python 解释器路径
python3 --version      # 检查 Python 版本
python3 -c "import sys; print(sys.executable)"

```

### 4.4.2. Python 包管理

```

pip3 list | grep -E "es-py|pyessdk-eswin-sample" # 查看已安装的 Python 包
pip3 show es_py                                # 查看包详细信息
pip3 show pyessdk-eswin-sample                  # 查看示例包详细信息
pip3 install --force-reinstall es_py-*.whl      # 强制重新安装
pip3 uninstall es_py                            # 卸载包

```

### 4.4.3. 系统状态检查

```

dpkg -l | grep es-sdk          # 检查 SDK DEB 包安装状态
/sbin/ldconfig -p | grep libes_ # 检查库链接状态
lsmod | grep es_                # 检查内核模块
ls /dev/es_*                    # 检查设备文件

```

### 4.4.4. 快速测试命令

```

# 激活环境并运行基础内存测试
python3 -c "import es_py; print('es_py 可用')" && \
python3 -m pyessdk_eswin_sample.memory_sample.memory_test

```